Lucas Augusto da Silva Cardoso (202200332300)

DCA3703 | Programação Paralela | Relato de experimento

TAREFA 2: TESTE DE DESEMPENHO COM MULTIPLICAÇÃO DE MATRIZES

Este documento detalha a abordagem metodológica e a solução implementada para o Experimento Computacional proposto na disciplina de Programação Paralela, em 13/08/2026.

1. EXPERIMENTO

"Implemente duas versões da multiplicação de matriz por vetor (MxV) em C: uma com acesso à matriz por linhas (laço interno variando a coluna) e outra por colunas (laço interno variando a linha). Meça o tempo de execução de cada versão usando uma função apropriada e execute testes com matrizes de diferentes tamanhos.

Identifique a partir de que tamanho os tempos passam a divergir significativamente e explique por que isso ocorre, relacionando suas observações ao uso da memória cache e ao padrão de acesso à memória."

2. CÓDIGO

O código implementa um ambiente de testes para avaliar o desempenho da multiplicação Matriz-Vetor em C. Para ter controle total sobre a memória, a matriz é alocada dinamicamente como um único vetor 1D contínuo, sendo acessada pela fórmula "i * N + j" e preenchida com valores previsíveis para facilitar a conferência dos cálculos.

Em seguida, as duas funções de multiplicação são partes essenciais para o experimento. A primeira versão lê a matriz por linhas (laço interno variando a coluna), seguindo a ordem natural e sequencial da linguagem C. A segunda versão

inverte os laços e lê por colunas, o que obriga o processador a dar longos "saltos" na memória a cada leitura.

Para medir o tempo com precisão de nanossegundos e sem interferências do sistema operacional, o código utiliza a função clock_gettime com a constante “CLOCK_MONOTONIC”. Por fim, a função principal orquestra tudo: ela aceita um tamanho específico de matriz pelo terminal ou executa automaticamente uma lista que cresce de 10 a 30.000. Essa progressão foi desenhada justamente para testar o limite e esgotar a memória cache da CPU aos poucos.

Figura 1: Código utilizado para testes

```c
#define _POSIX_C_SOURCE 199309L // Necessário para clock_gettime() no Linux
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>

// Recuperar tempo de relógio transcorrido (em segundos) conforme orientado na sala de aula
double recuperar_tempo_instantaneo() {
    struct timespec ts; // Estrutura para armazenar o tempo
    clock_gettime(CLOCK_MONOTONIC, &ts); // Obtém o tempo de relógio monotônico (não retrocede)
    return (double)ts.tv_sec + (double)ts.tv_nsec * 1e-9; // Converte para segundos
}

// Versão A: Acesso por linha (loop interno varia colunas) - Padrão do C
// O * antes do parametro indica que é um ponteiro para o tipo double, ou seja, é um array de doubles
void multiplicacao_matriz_vetor_linha(const double *A, const double  *x, double  *y, int N) {
    for (int i = 0; i < N; i++) {
        double soma = 0.0;
        for (int j = 0; j < N; j++) {
            soma += A[i * N + j] * x[j];
        }
        y[i] = soma;
    }
}

// Versão B: Acesso por coluna (loop interno varia linhas) - Estrutura de coluna
void multiplicacao_matriz_vetor_coluna(const double *A, const double *x, double *y, int N) {
    for (int i = 0; i < N; i++) {
        y[i] = 0.0;
    }
    for (int j = 0; j < N; j++) {
        double xj = x[j];
        for (int i = 0; i < N; i++) {
            y[i] += A[i * N + j] * xj;
```

```
        }
    }
}

// Inicializa matriz e vetor com valores determinísticos para facilitar a verificação de resultados
void inicializar_matrizes(double *matriz_A, double *vetor_x, int N) {
    for (int i = 0; i < N; i++) {
        vetor_x[i] = (double)i / N;
        for (int j = 0; j < N; j++) {
            matriz_A[i * N + j] = (double)(i + j) / N;
        }
    }
}



void realizar_teste(int N) {
    printf("Calculando com N = %d...\n", N);

    // Alocando memória
    double *matriz_A = (double *)malloc((size_t)N * N * sizeof(double));
    double *vetor_x = (double *)malloc((size_t)N * sizeof(double)); // o vetor_x seria
    double *resultado_em_linha = (double *)malloc((size_t)N * sizeof(double));
    double *resultado_em_coluna = (double *)malloc((size_t)N * sizeof(double));

    inicializar_matrizes(matriz_A, vetor_x, N);

    double inicio_calculo_vetor_linha = recuperar_tempo_instantaneo();
    multiplicacao_matriz_vetor_linha(matriz_A, vetor_x, resultado_em_linha, N);
    double fim_calculo_vetor_linha = recuperar_tempo_instantaneo();
    double tempo_total_calculo_vetor_linha = fim_calculo_vetor_linha - inicio_calculo_vetor_linha;

    double inicio_calculo_vetor_coluna = recuperar_tempo_instantaneo();
    multiplicacao_matriz_vetor_coluna(matriz_A, vetor_x, resultado_em_coluna, N);
    double fim_calculo_vetor_coluna = recuperar_tempo_instantaneo();
    double tempo_total_calculo_vetor_coluna = fim_calculo_vetor_coluna - inicio_calculo_vetor_coluna;

    printf("  Linha:       %10.6f s\n", tempo_total_calculo_vetor_linha);
    printf("  Coluna:      %10.6f s\n", tempo_total_calculo_vetor_coluna);

    free(matriz_A);
    free(vetor_x);
    free(resultado_em_linha);
    free(resultado_em_coluna);
    printf("\n");
}

int main(int argc, char *argv[]) {
    if (argc > 1) {
```

```
    // Rodando com um tamanho de matriz específico fornecido como argumento
    int N = atoi(argv[1]);
    if (N <= 0) {
        fprintf(stderr, "Tamanho inválido:  %s\n", argv[1]);
        return 1;
    }
    realizar_teste(N);

    return 0;
}

//Testando diferentes tamanhos de matriz
int tamanhos[] = {10, 50, 100, 250, 500, 1000, 2000, 5000, 10000, 15000, 20000, 25000};
int quantidade_tamanhos = 12;

for (int i = 0; i < quantidade_tamanhos; i++) {
    realizar_teste(tamanhos[i]);
}
return 0;
}
```

3. RESULTADOS

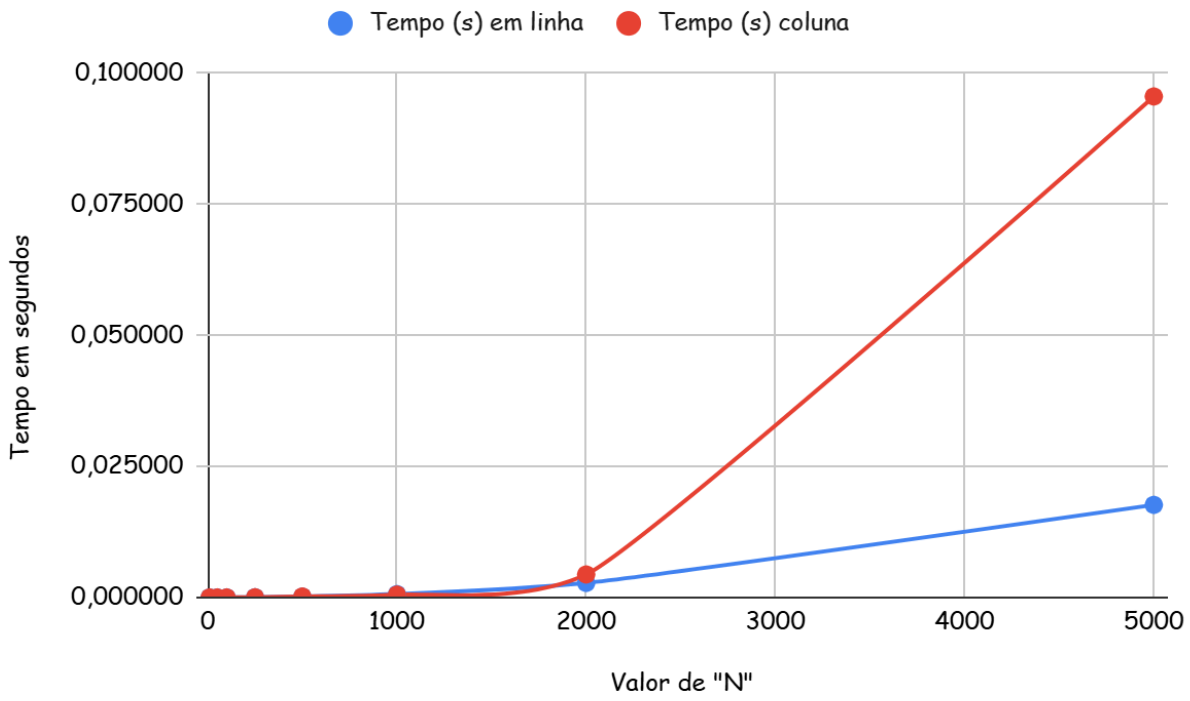
O resultado dos testes do programa gerado pelo código acima, podem ser vistos na tabela (Tabela 1) e gráfico abaixo (Figura 2).

Tabela 1: Dados do experimento

Valor de "N"	Tempo (s) em linha	Tempo (s) coluna
10	0,000000	0,000002
50	0,000001	0,000001
100	0,000006	0,000005
250	0,000038	0,000036
500	0,000160	0,000183
1000	0,000642	0,000547
2000	0,002776	0,004391
5000	0,017667	0,095587
10000	0,070259	0,465758

15000	0,158004	1,054877
20000	0,280120	1,925081
25000	0,438676	3,048347

Figura 2: Evolução do tempo em linha e coluna em quantidade de iterações



A análise dos resultados demonstra claramente o impacto da memória cache no desempenho do código. Para matrizes de até N = 1000, os tempos de execução das duas versões são praticamente idênticos. Isso ocorre porque a matriz é pequena o suficiente para caber inteiramente na cache da CPU, mascarando a ineficiência da leitura por colunas graças à velocidade altíssima desse componente.

O computador testado possui 32 MB de cache L3. Considerando que cada número double ocupa 8 bytes em C, a capacidade máxima da cache comporta uma matriz quadrada de até N ≈ 2048 (conforme o cálculo N^2 x 8 = 33.554.432 bytes). Os dados tabelados refletem essa matemática com exatidão: os tempos de execução começam a divergir significativamente logo após N = 2000.

Essa divergência ocorre devido ao padrão nativo da linguagem C, que armazena os elementos fisicamente em linha na memória (row-major order). A

versão que acessa a matriz por linhas acompanha esse fluxo natural, garantindo máxima eficiência na leitura. Já a versão por colunas é obrigada a realizar longos saltos de memória. Ao estourar o limite de 32 MB da cache, esses saltos contínuos geram milhares de erros de leitura (cache misses), forçando a CPU a buscar os dados de forma fragmentada e desordenada na memória RAM, que é exponencialmente mais lenta.

O peso desse gargalo de memória fica evidente nos testes maiores. No cenário extremo da tabela, a execução por linha finalizou em apenas 0,43 segundos, enquanto a versão por colunas levou mais de 3,04 segundos.

4. CONSIDERAÇÕES FINAIS

Os resultados deste experimento deixam claro que apenas escrever um código que funciona e entrega a matemática correta não é suficiente para garantir um bom desempenho. Ao longo dos testes da multiplicação Matriz-Vetor, foi possível ver na prática que a forma como o algoritmo percorre os dados muda tudo. Quando o código tentou acessar a matriz por colunas, a organização da memória cache foi completamente quebrada. Isso forçou a CPU a buscar as informações na lenta memória RAM de maneira desordenada, gerando um gargalo enorme que multiplicou o tempo de execução da tarefa.

Dessa forma, pode-se se dizer que desenvolver software de alto desempenho exige uma verdadeira sintonia entre o código e a máquina. É fundamental que o programador conheça as particularidades da linguagem que está utilizando (como o fato de a linguagem C armazenar seus dados nativamente em linhas contínuas) e compreenda os limites físicos do hardware em que está trabalhando, como a capacidade da memória cache.

Assim sendo, sem alinhar a lógica de leitura do algoritmo com a forma que o computador realmente organiza os dados fisicamente, grande parte do poder de processamento da máquina é desperdiçada. Respeitar essa arquitetura é o primeiro e mais importante passo antes de tentar aplicar qualquer outra técnica avançada de otimização.